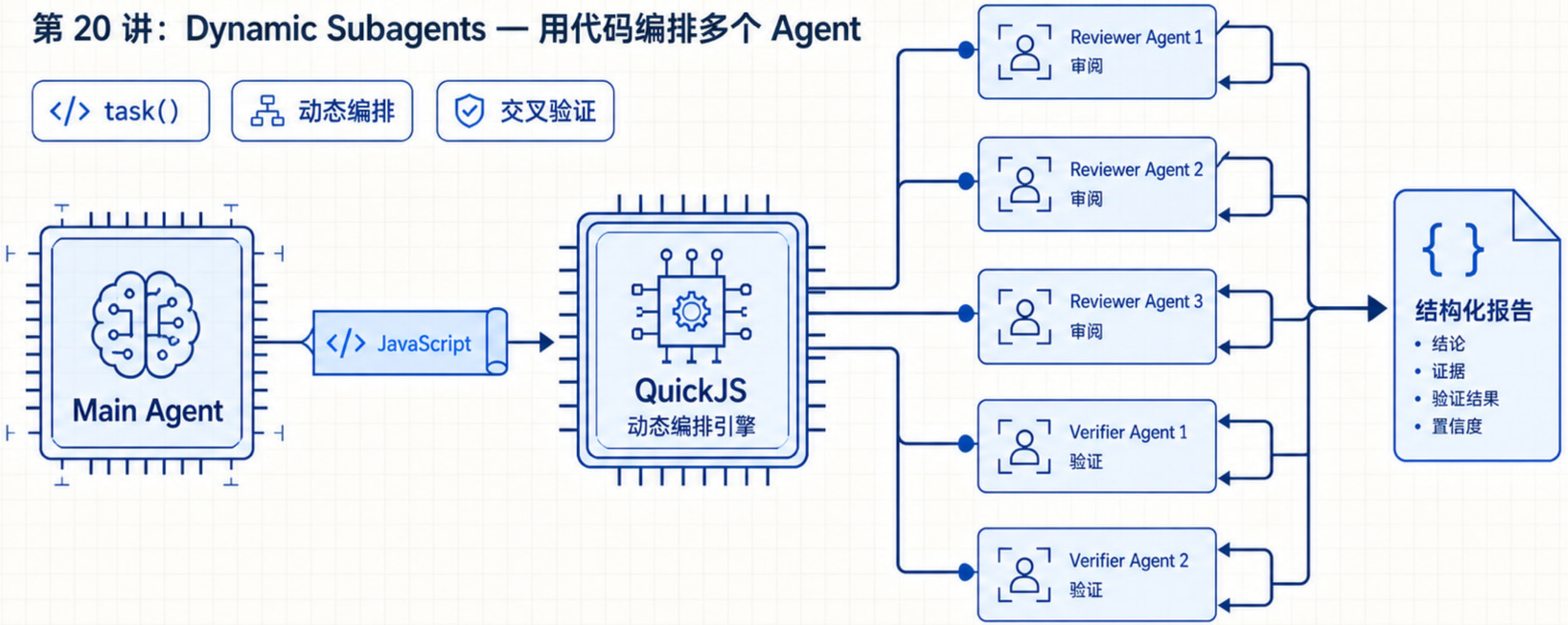
Deep Agents 实战

第 20 讲：Dynamic Subagents — 用代码编排多个 Agent

`</> task()` 动态编排 交叉验证



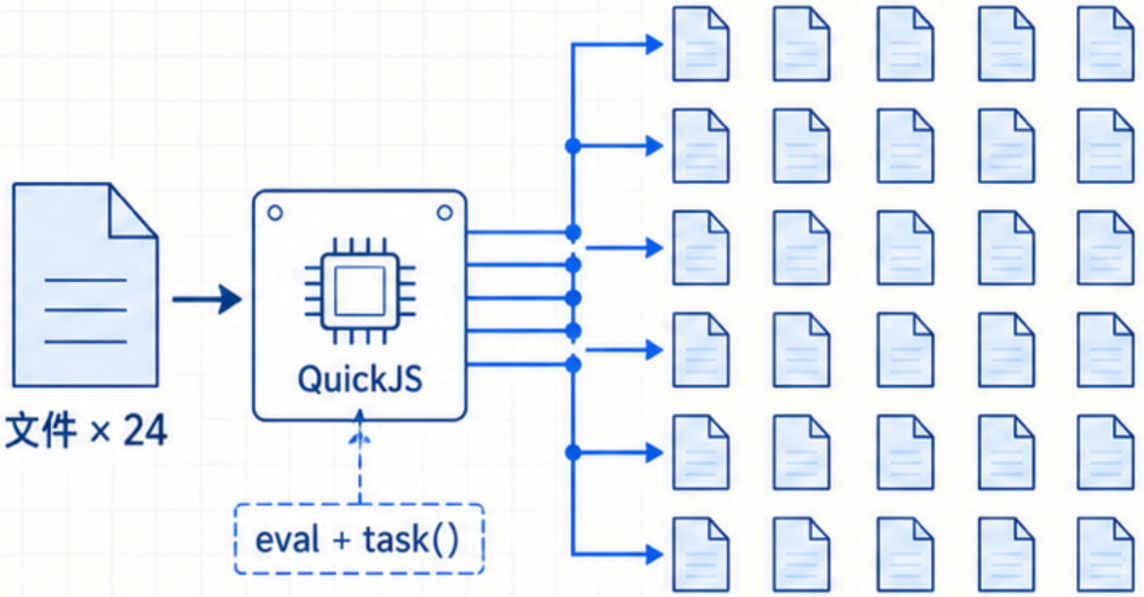
一个委派很清楚，24 个委派需要编排

逐次选择会把覆盖范围交给模型记忆

逐次委派

漏项与额外轮次

动态编排



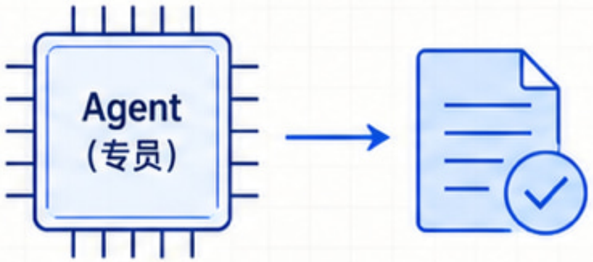
代码保证覆盖

先看任务形状，再选择子 Agent 机制

单次委派、批量编排和后台生命周期不是一回事

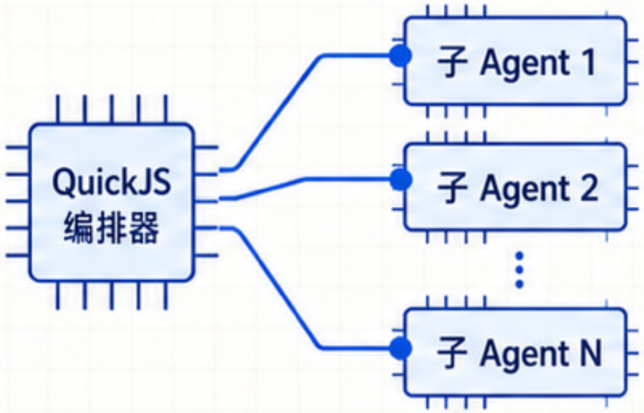
任务怎样展开？

单个明确任务 → 普通 task



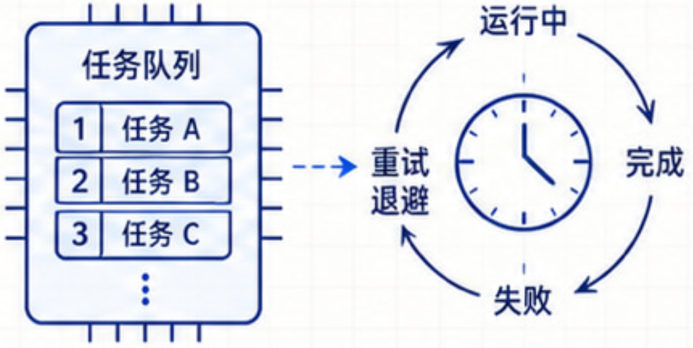
专业 Agent
（单一专长）

批量 / 多阶段 → Dynamic Subagents



并行 / 条件 / 顺序编排

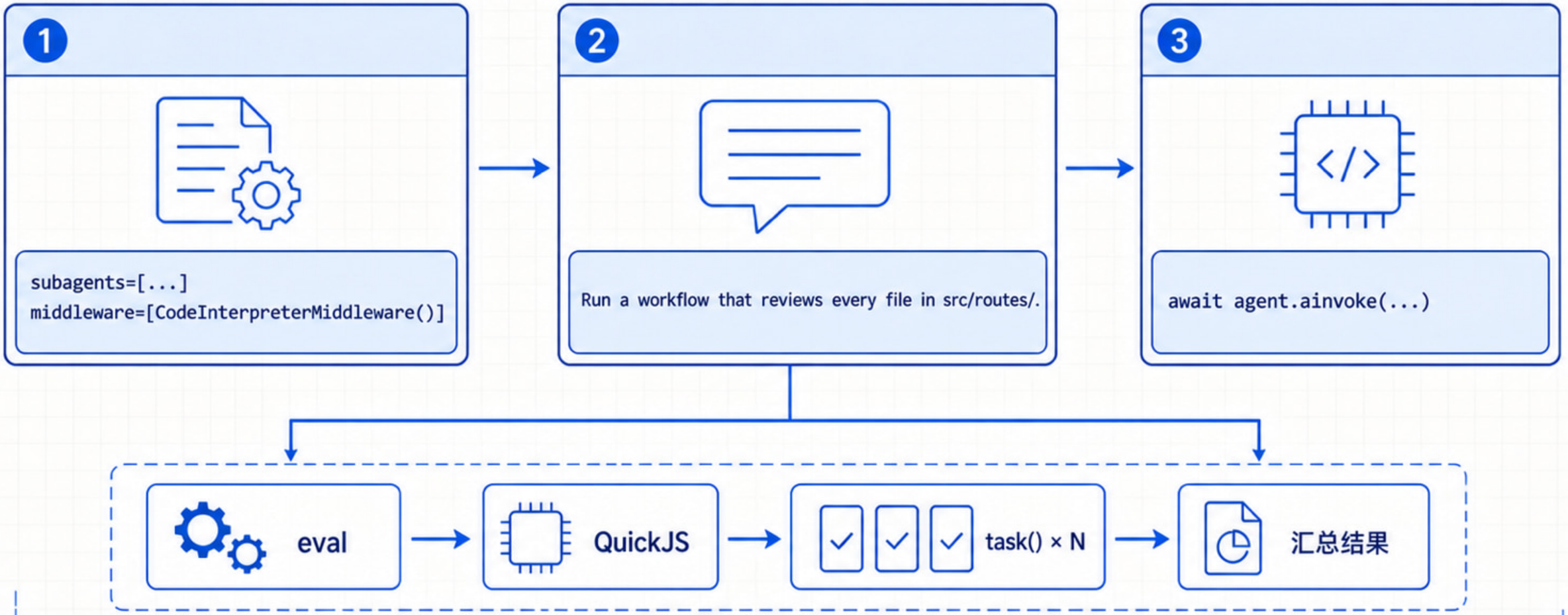
后台持续运行 → Async Subagents



异步执行 + 生命周期管理

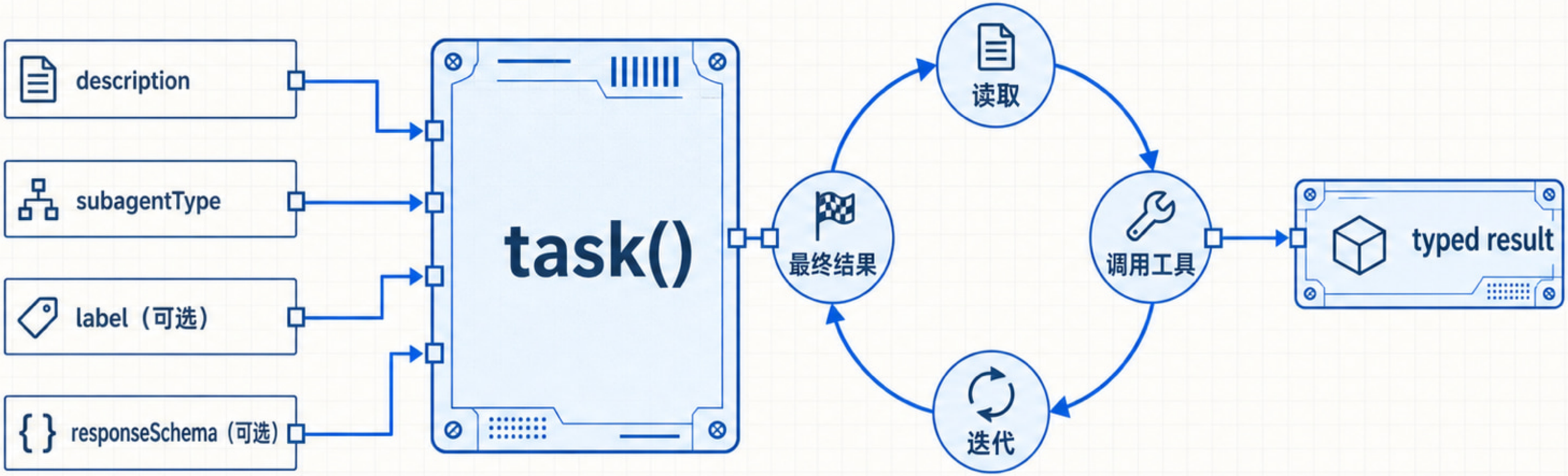
动态工作流由配置、提示词和调用共同触发

“workflow”是提示信号，不是新的 API 参数



task() 不是一次模型调用

每次调度都会运行一个完整子 Agent 循环



name、description、system_prompt 定义一个子智能体

name 是 subagentType 的稳定值

reviewer

"name": "reviewer"

"description": "发现候选问题"

"system_prompt": "报告 file / line / severity"

verifier

"name": "verifier"

"description": "独立确认或反驳"

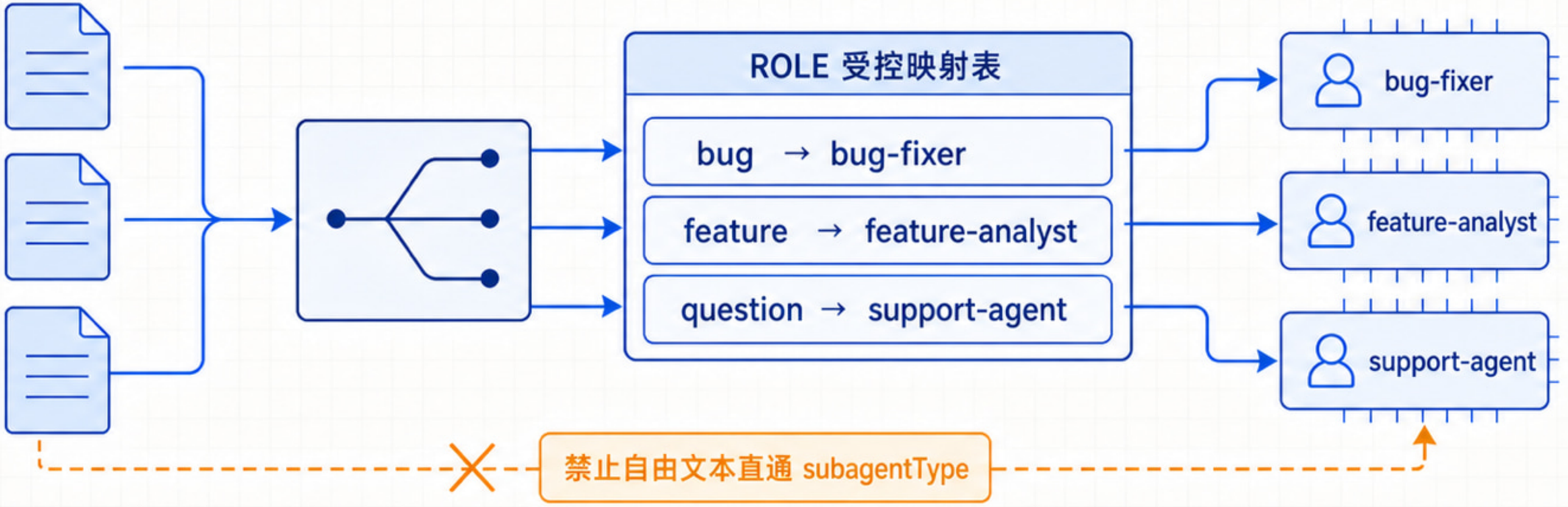
"system_prompt": "只确认真实问题"

subagents=[reviewer, verifier]

description 帮助选择角色，不代表权限

场景 1：混合工单先分类，再固定映射角色

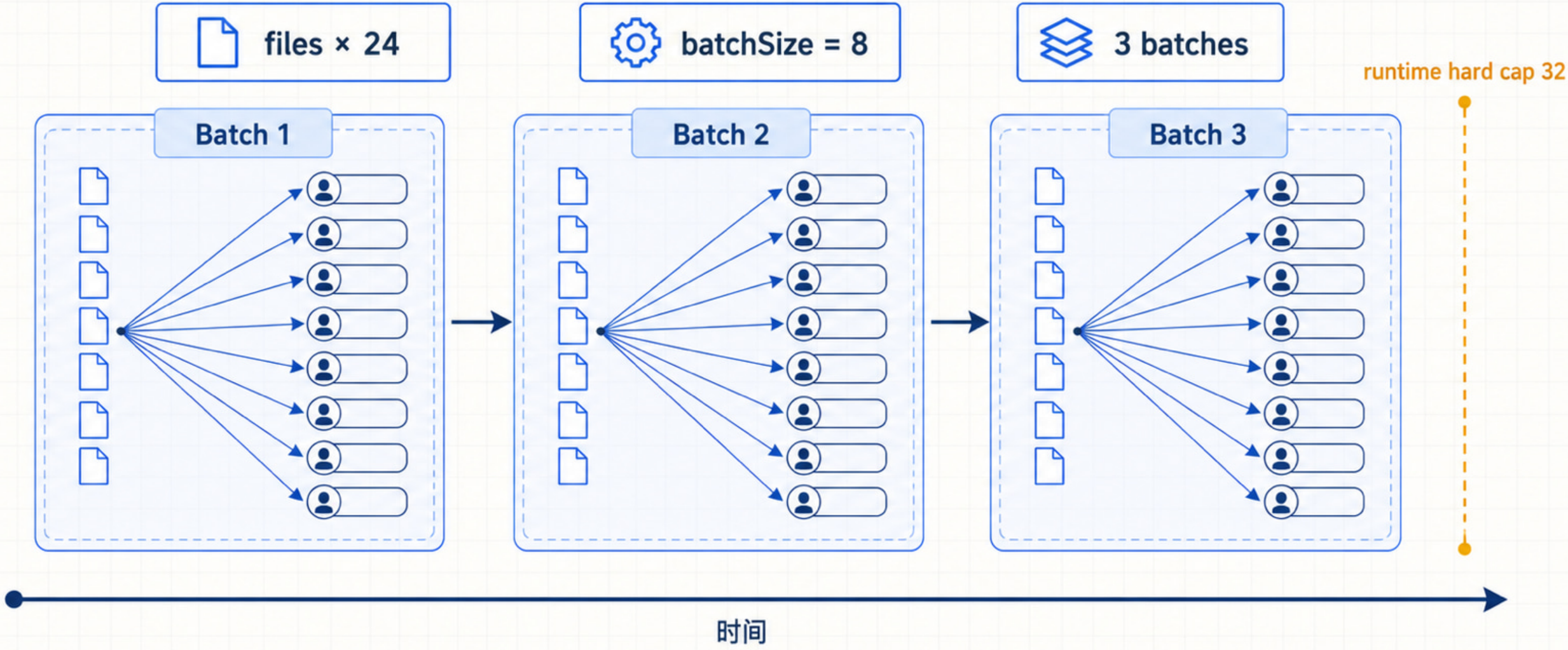
分类结果只能通过受控映射表选择 subagentType




```
const ROLE = { bug: "bug-fixer", feature: "feature-analyst", question: "support-agent" };
task({ description: ticket.text, subagentType: ROLE[ticket.category] })
```

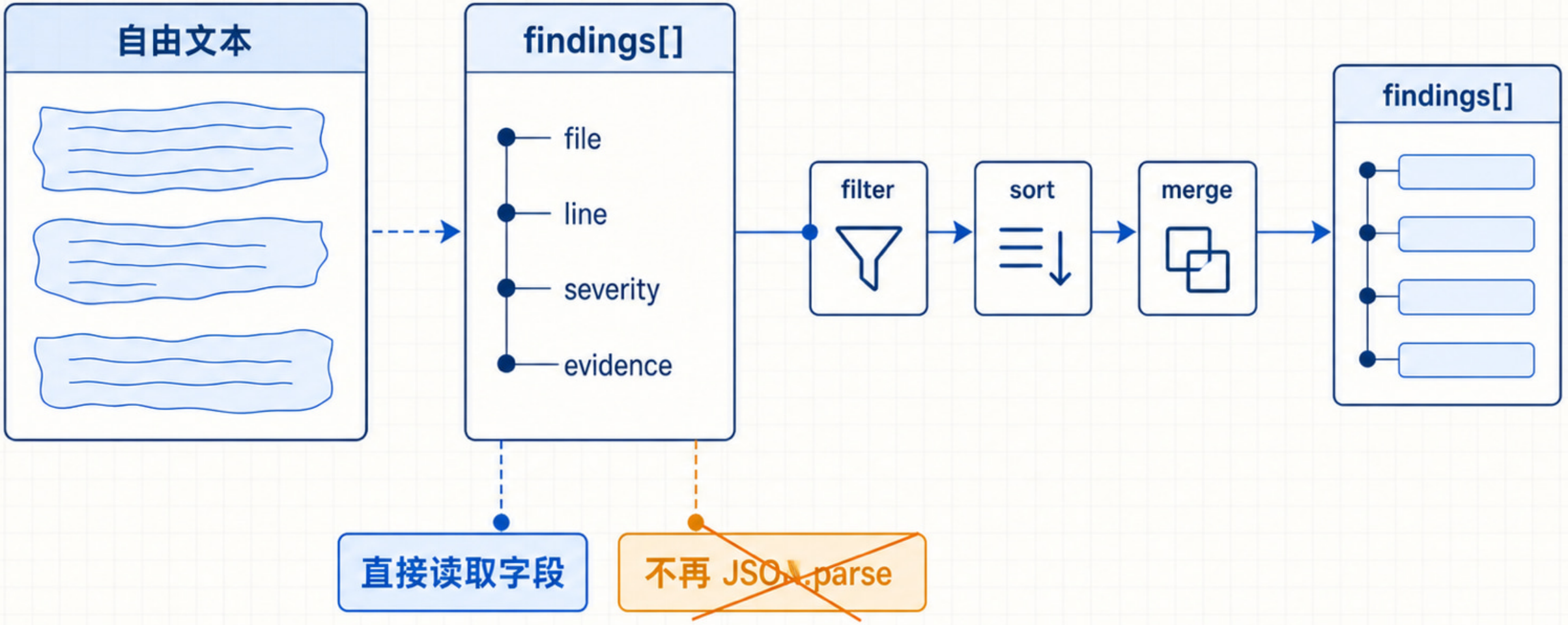
并行扇出：覆盖全部输入，但要分批

每批 8 个，Promise.all，批次串行推进



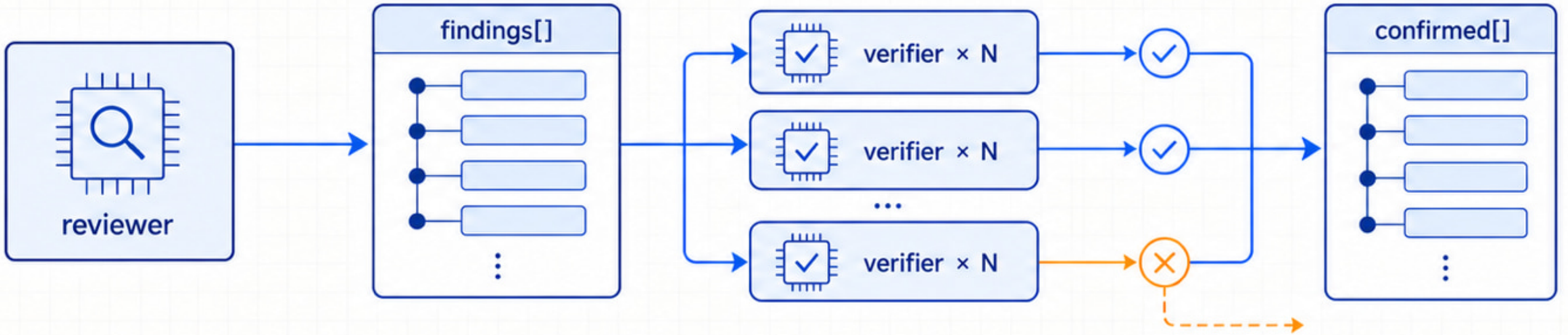
responseSchema 是阶段之间的接口

结果已经是 typed JavaScript value



场景 2：先发现，再独立复核

reviewer 偏召回，verifier 偏准确



1

```
const { findings } = await task({ subagentType:
"reviewer", responseSchema: findingsSchema });
```

2

```
task({ subagentType: "verifier",
responseSchema: verdictSchema })
```

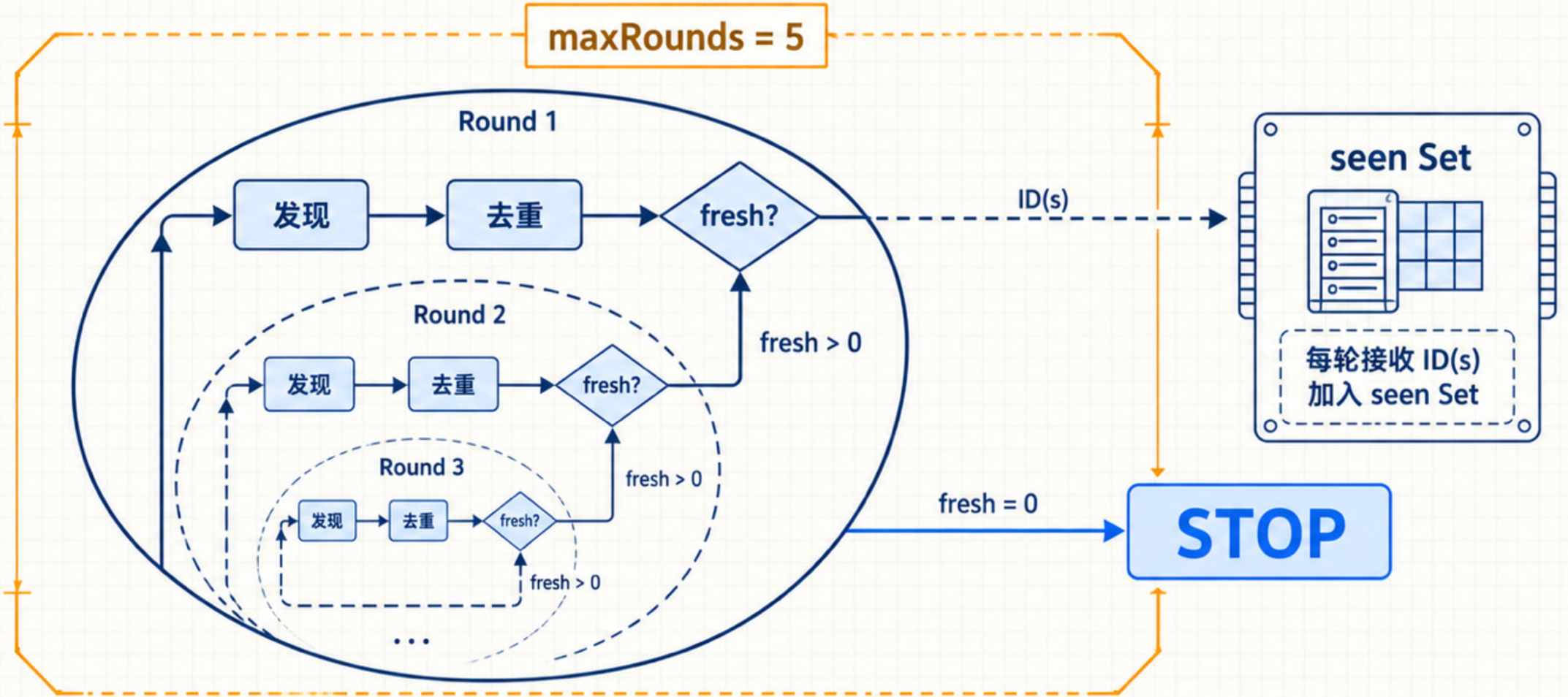
3

```
findings.filter((_, i) => verdicts[i]?.confirmed)
```

responseSchema 返回 typed value，无需 JSON.parse

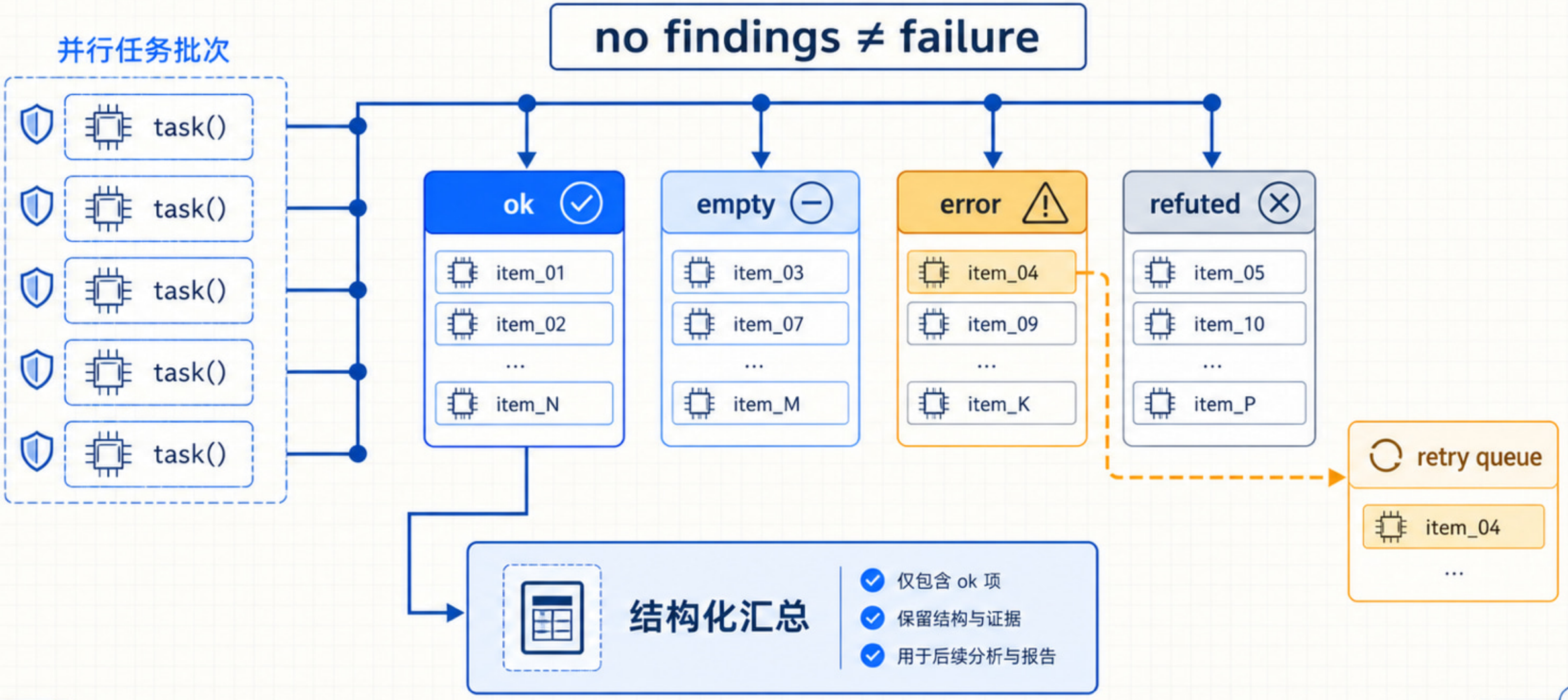
迭代收敛：没有新增就停止

业务停止条件之外，还要设置最大轮数



单项失败不该抹掉整批成功结果

ok、empty、error、refuted 必须分开记录



把预算、状态、审批和权限放在同一张边界图

四条边界共同约束 workflow

预算

- batchSize
- max_ptc_calls

状态

- mode
- Checkpointer

执行边界

审批

- parent interrupt_on
  不逐次覆盖 task()

权限

- 最小 tools / permissions

失败隔离、重试和降级写进 workflow

让代码管控制流，让 Agent 管判断

Dynamic Subagents 上线前四项检查